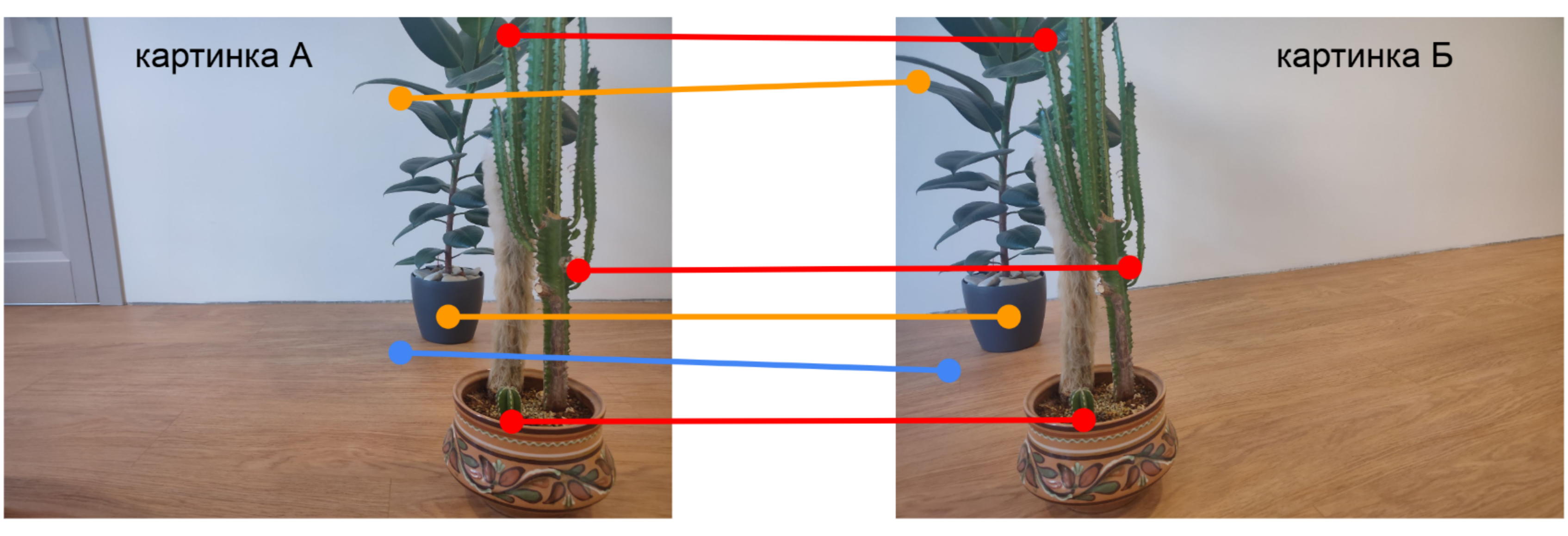
🌐https://youtu.be/rEF0zkv2cn8?si=3KdYvDcsMLA501Dx

Сопоставляем не каждые точки, а только "хорошие".

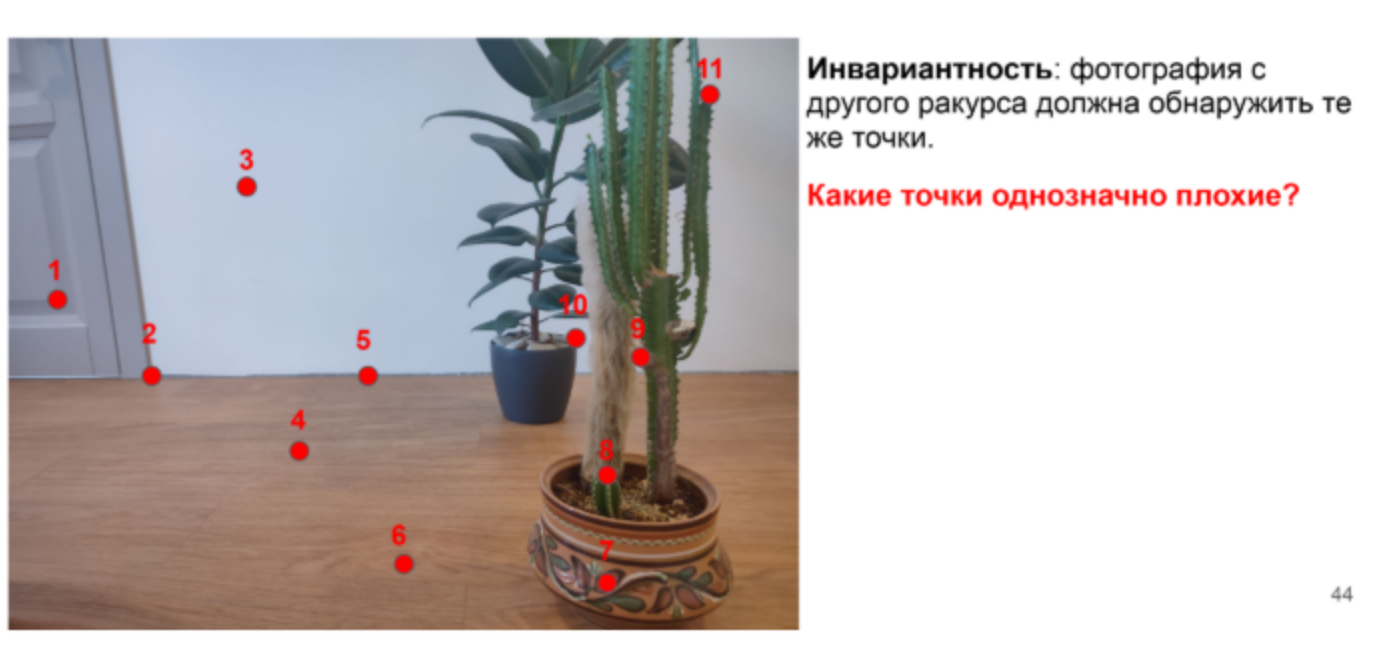
Как сопоставлять? Возьмем окрестность - посчитаем цвета и посмотрим на похожие.

Задача сопоставления фотографий:
1) Как сопоставить точки? Как алгоритму понять где одно и то же?
2) Если свести задачу к выбору ключевых точек - какие к ним требования?
3) К каким искажениям должны быть устойчивы (инварианты) точки? - поворот, свдиг камеры, перспективное искажения, перепад яркости + инвариантность к шумам (быть устойчивым ко всему)
Должна быть "шиза" - один ломает, другой чинит
4) Как сопоставить точки? Как оценить похожесть точек? - по окрестности цветов (например)



2 логические части:

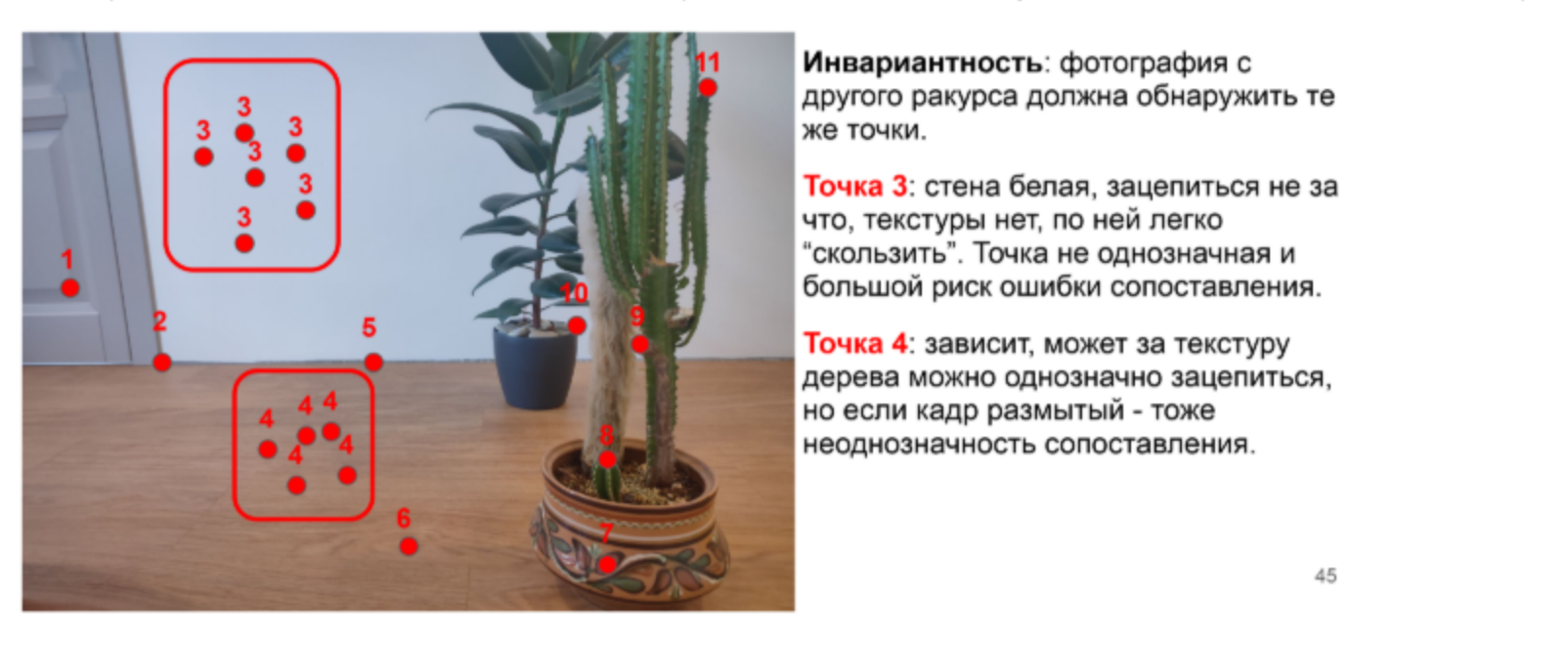
- Detector - инвариантно к искажениям выбрать ключевые точки
- Descriptor - построить для каждой точки описания в виде вектора чисел



Инвариантность: фотография с другого ракурса должна обнаружить те же точки.

Какие точки однозначно плохие?

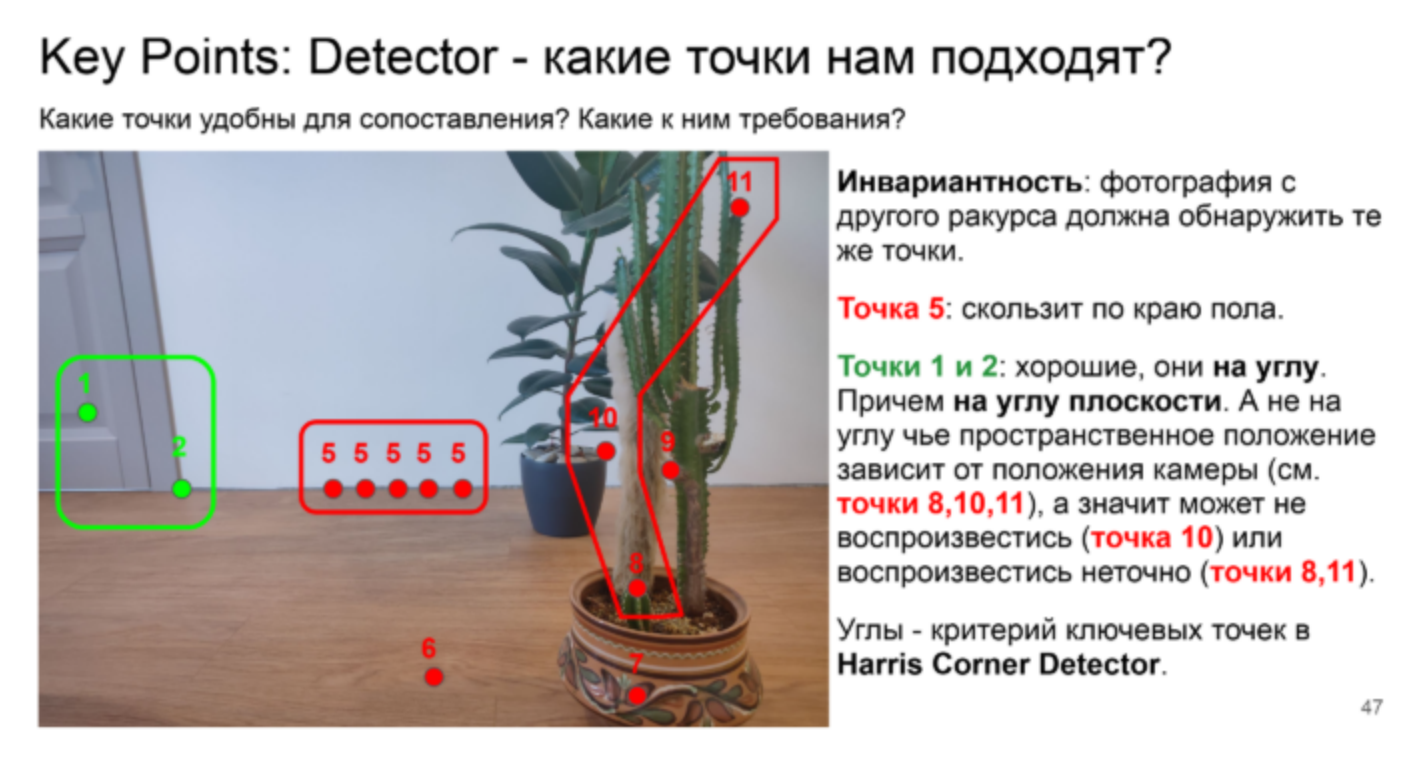
Плохие точки: 3, 4 (но не факт, текстура не однозначна, при достаточном удалении - пол станет однородным. Но если достаточно приблизиться, то будет четкое качество поверхности (выемки и тд)



Инвариантность: фотография с другого ракурса должна обнаружить те же точки.

Точка 3: стена белая, зацепиться не за что, текстуры нет, по ней легко "скользить". Точка не однозначная и большой риск ошибки сопоставления.

Точка 4: зависит, может за текстуру дерева можно однозначно зацепиться, но если кадр размытый - тоже неоднозначность сопоставления.



Key Points: Detector - какие точки нам подходят?

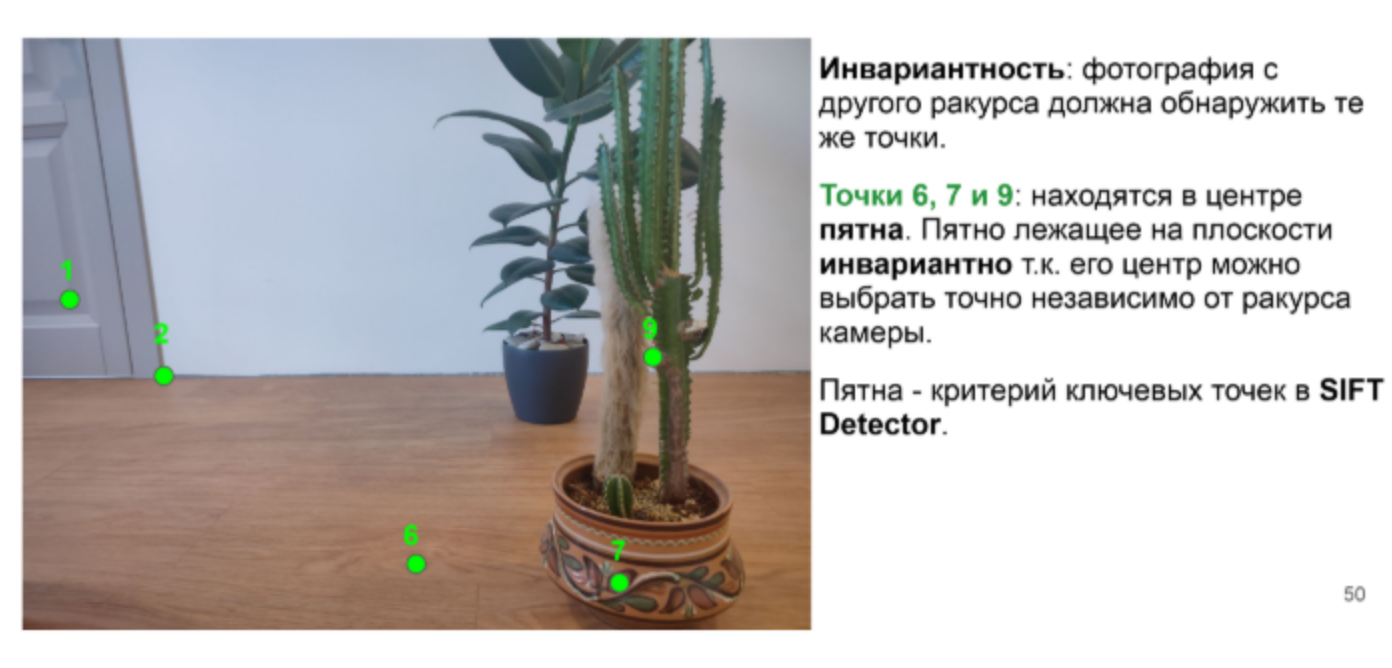
Какие точки удобны для сопоставления? Какие к ним требования?

Инвариантность: фотография с другого ракурса должна обнаружить те же точки.

Точка 5: скользит по краю пола.

Точки 1 и 2: хорошие, они **на углу**. Причем **на углу плоскости**. А не на углу чье пространственное положение зависит от положения камеры (см. **точки 8,10,11**), а значит может не воспроизвестись (**точка 10**) или воспроизвестись неточно (**точки 8,11**).

Углы - критерий ключевых точек в **Harris Corner Detector**.



Инвариантность: фотография с другого ракурса должна обнаружить те же точки.

Точки 6, 7 и 9: находятся в центре **пятна**. Пятно лежащее на плоскости **инвариантно** т.к. его центр можно выбрать точно независимо от ракурса камеры.

Пятна - критерий ключевых точек в **SIFT Detector**.

точка 2 может быть немного плохая, так как зависит от глубины, может быть ближе, а может быть дальше

точка 10 зависит от положения камеры - она всегда угловая, но положение в пространстве будет меняться
точка 8 и 11 тоже могут "скользить" при повороте камер

Поэтому точки должны быть на плоскости и уникально идентифицироваться

Поэтому ищем углы - Harris Corner Detector

Desctiptor

Как сопоставлять точки между двумя картинками? Строим описание для них.
SIFT Descriptor: 128 float = 512 bytes

Чтобы сравнить две точки, берем два вектора используем метрику, можно взять евклидово расстояние

$$\mathrm{dist}(A,B) = \sqrt{\sum_{i=1}^{128} (A_i - B_i)^2}$$

Но должна быть инвариантность Descriptor'а чтобы каждый индекс описывал одну черту, а то как i-ый A и i-ый B одинаковые элементы

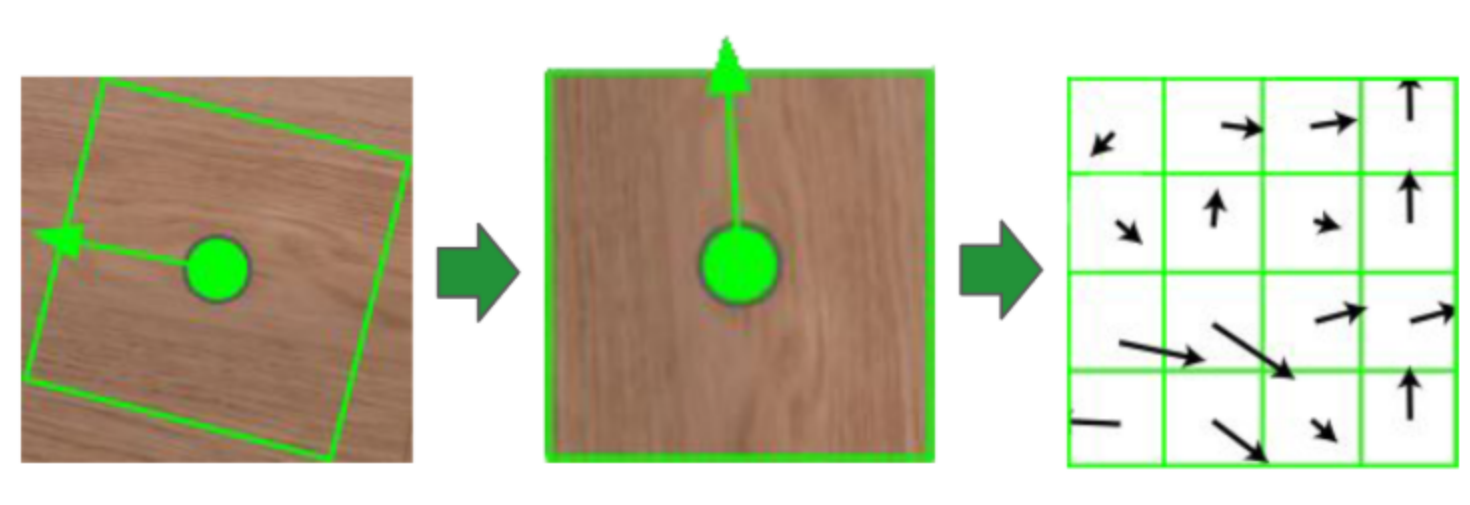
Еще может навредить освещение, если мы будем записывать только цвета (инвариантность по описанию)
Еще сдвиг и масштаб (квадратик должен быть примерно одинакового размера)

Если взять цвета (пиксели) вокруг точки, то при повороте будут уже другие пиксели, поэтому квадрат вокруг точки - неинвариантный к повороту дескриптора.

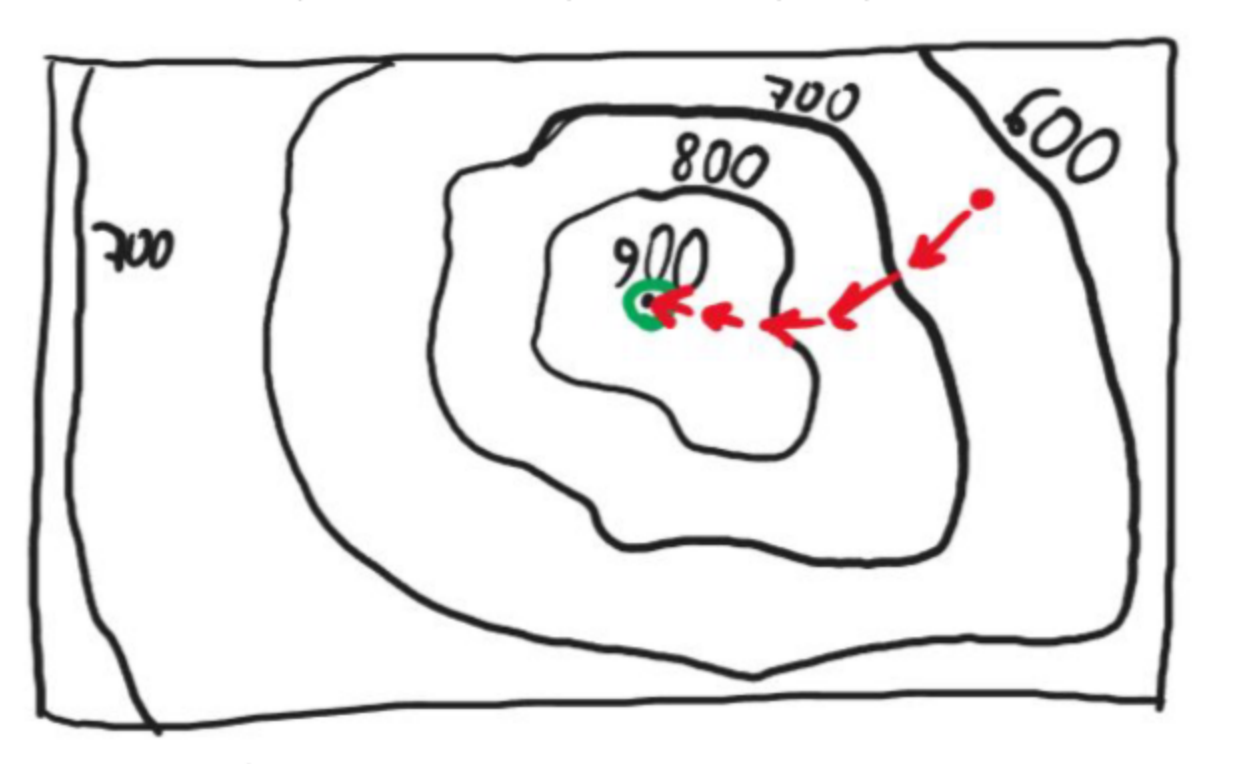
Если перебирать все набор пикселей вокруг и минимизировать ошибку, то долго

Нужна какая-то функция (градиент), в каждой точке указываем "направление гравитации" (какая-то стрелочка для обобщения примера)
Например самое популярное направление градиента в локальной окрестности

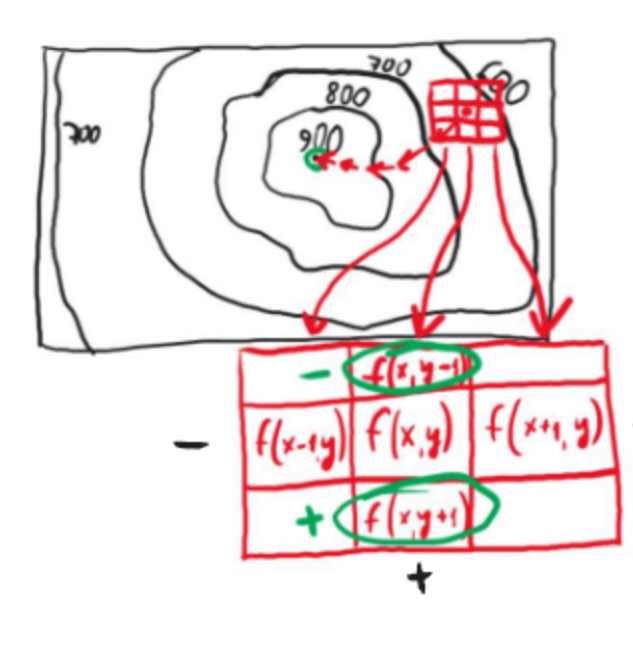
Пусть уже нормализовано всё (Оракул дал, как нужно повернуть, чтобы получить инвариант - упрощение для объяснения)



Вычисление градиента в картинке оператором Собеля



Я хочу забраться на гору, для этого иду в направлении, где земля становится выше, так я могу упасть в локальный экстремум (сойтись в локальный экстремум), но пусть я поброжу где-то еще и нивелирую это



$$\bar{f}' = (f'_x(x,y), f'_y(x,y))$$

направление, в котором сильнее всего меняется функция

сильное направление по Ox
сильное направление по Oy

$$f'_x(x,y) = f(x+1,y) - f(x-1,y)$$
$$f'_y(x,y) = f(x,y+1) - f(x,y-1)$$

x, y

считаем производную в точке (x,y)

Как это посчитать? Как вариант использовать свертку

$$\begin{array}{|c|c|c|} \hline 0 & f(x,y-1) & 0 \\ \hline f(x-1,y) & f(x,y) & f(x+1,y) \\ \hline 0 & f(x,y+1) & 0 \\ \hline \end{array} \times \begin{pmatrix} -1 & \cdot & +1 \\ -2 & \cdot & +2 \\ -1 & \cdot & +1 \end{pmatrix} = \vec{f}_x$$

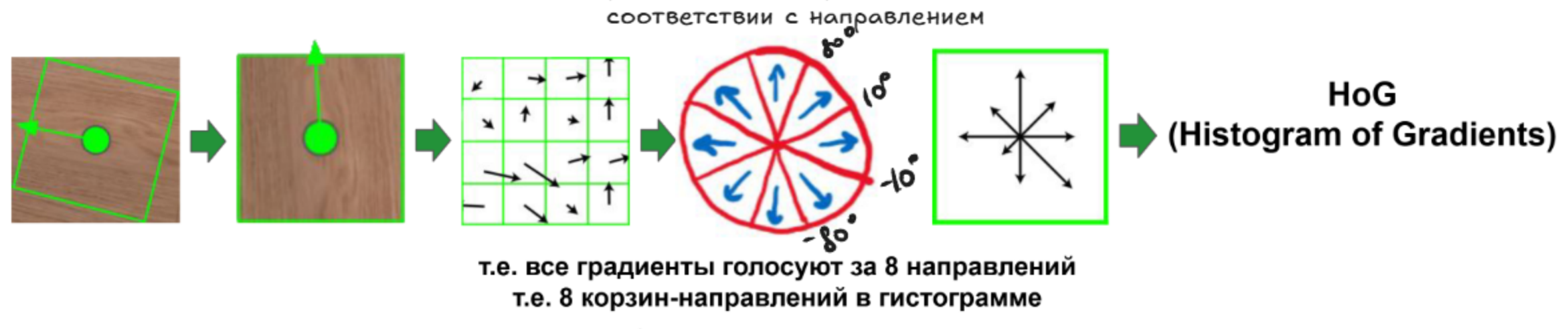
$$-1f(x-1,y-1) + 1f(x+1,y-1)$$
$$-2f(x-1,y) + 2f(x+1,y)$$
$$-1f(x-1,y+1) + 1f(x+1,y+1)$$

Но с начала лучше сделать размытие с помощью Гауссово ядра (для борьбы с шумом и выбросов)
Потом считаем уже производные

$$= \overset{\text{ядра Гаусса}}{\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}} \text{ или нормируем} = \begin{pmatrix} 1/16 & 1/8 & 1/16 \\ 1/8 & 1/4 & 1/8 \\ 1/16 & 1/8 & 1/16 \end{pmatrix}$$

HoG = N чисел: одно число - одно направление. Число получается суммой длин градиентов которые указывают в этом направлении.

разложили по корзинкам в соответствии с направлением

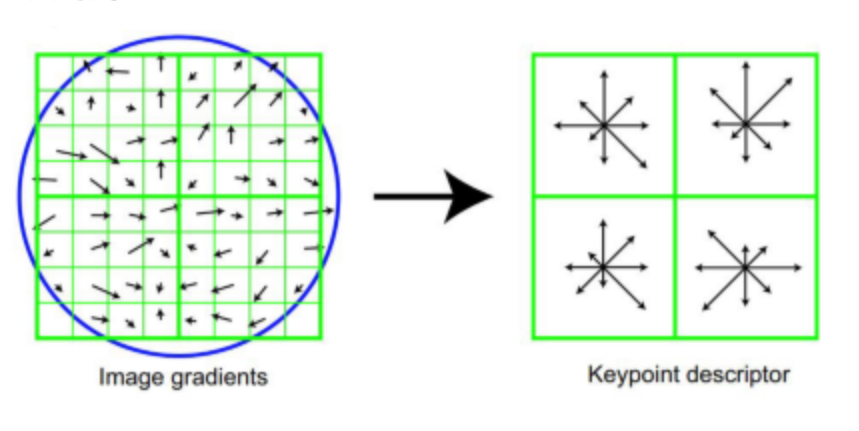


т.е. все градиенты голосуют за 8 направлений
т.е. 8 корзин-направлений в гистограмме

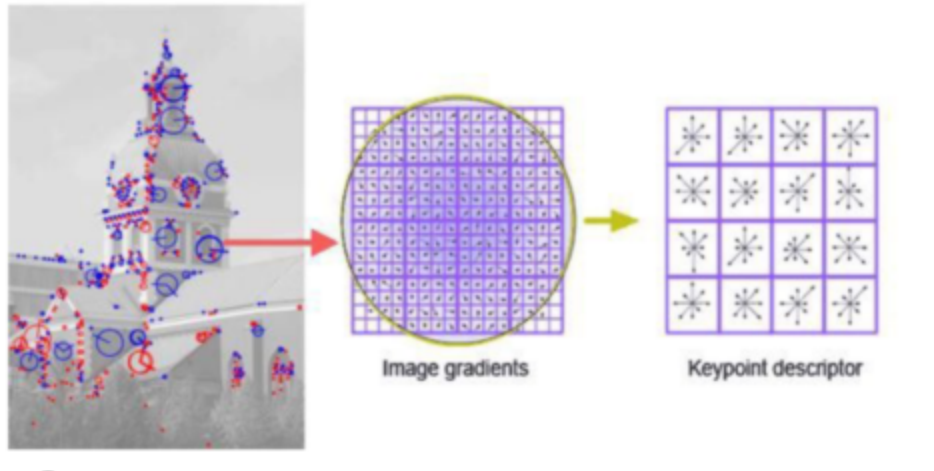
на выход 8 стрелок
на вход может быть любое (16, 32, 64, max = кол-во пикселей)

1) Инвариантность по повороту
2) Инвариантность по масштабу
3) Инвариантность по описанию:
   - К изменению яркости (если увеличить значения пикселей в 2 раза, тогда направления увеличатся в два раза - другой дескриптор, но если их нормировать, но всё будет хорошо)
   - К малому сдвигу

SIFT Descriptor = HoG, т.е. 128-и корзинная гистограмма. В каждой корзине записано насколько популярно это направление.
P.S. на самом деле локальный патч бьется на мини-патчи, и строится несколько HoG - для каждого мини-патча.



тут 32 числа, но это упрощение, в оригинальной статье (128)



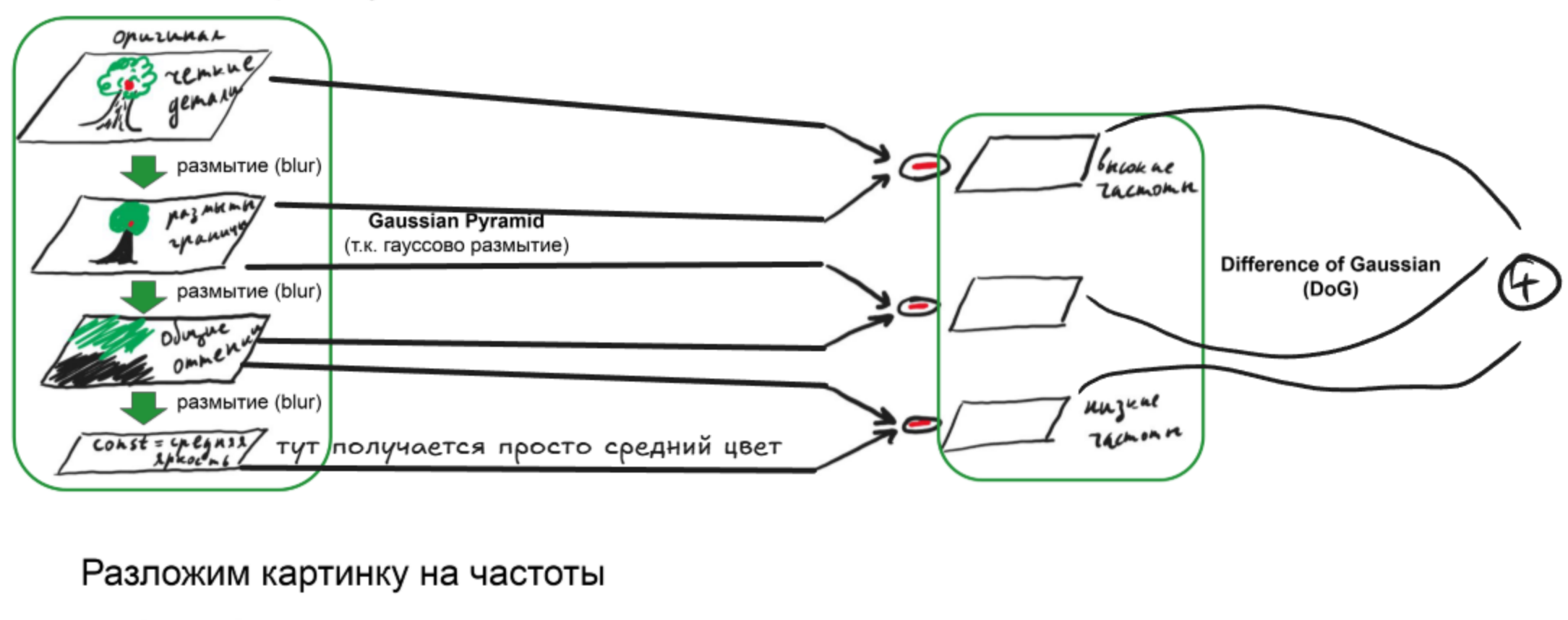
🌐https://medium.com/machine-learning-world/feature-extraction-and-similar-image-search-with-opencv-for-newbies-3c59796bf774

SIFT Key Points: Detector & Descriptor

1) Выбираем ключевыми точками "пятна"
2) У каждой точки инвариантно определяем:
   2.1) Поворот
   2.2) Масштаб
3) Для каждой точки строим дескриптор из HoG (гистограммы градиентов) по квадратной окрестности точки (просто квадратный патч 15x15 px?).
После чего определяем мажорирующее направление (главное)

На этапе детекции точек опредяем еще направление (поворот) и масштаб

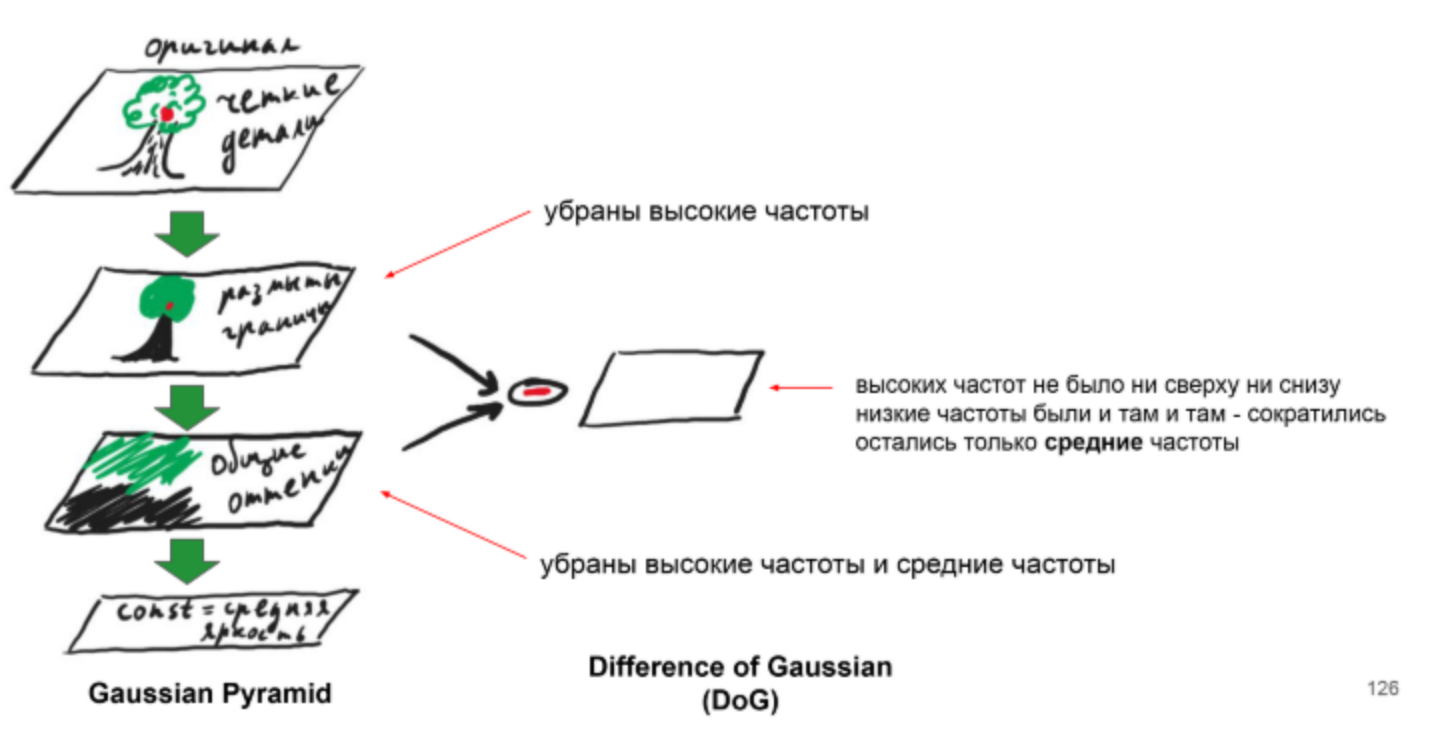
Разложим картинку на частоты



тут получается просто средний цвет

+ = Оригинал (четкие детали) (почти)
Нужно еще добавить низкие частоты, так как они были потеряны

Разложим картинку на частоты

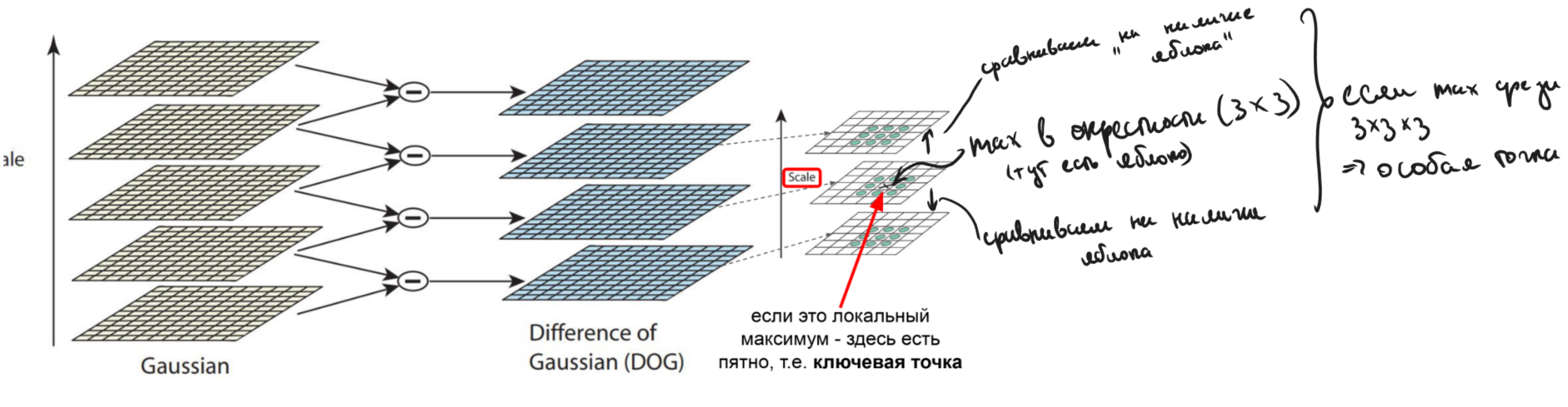


убраны высокие частоты

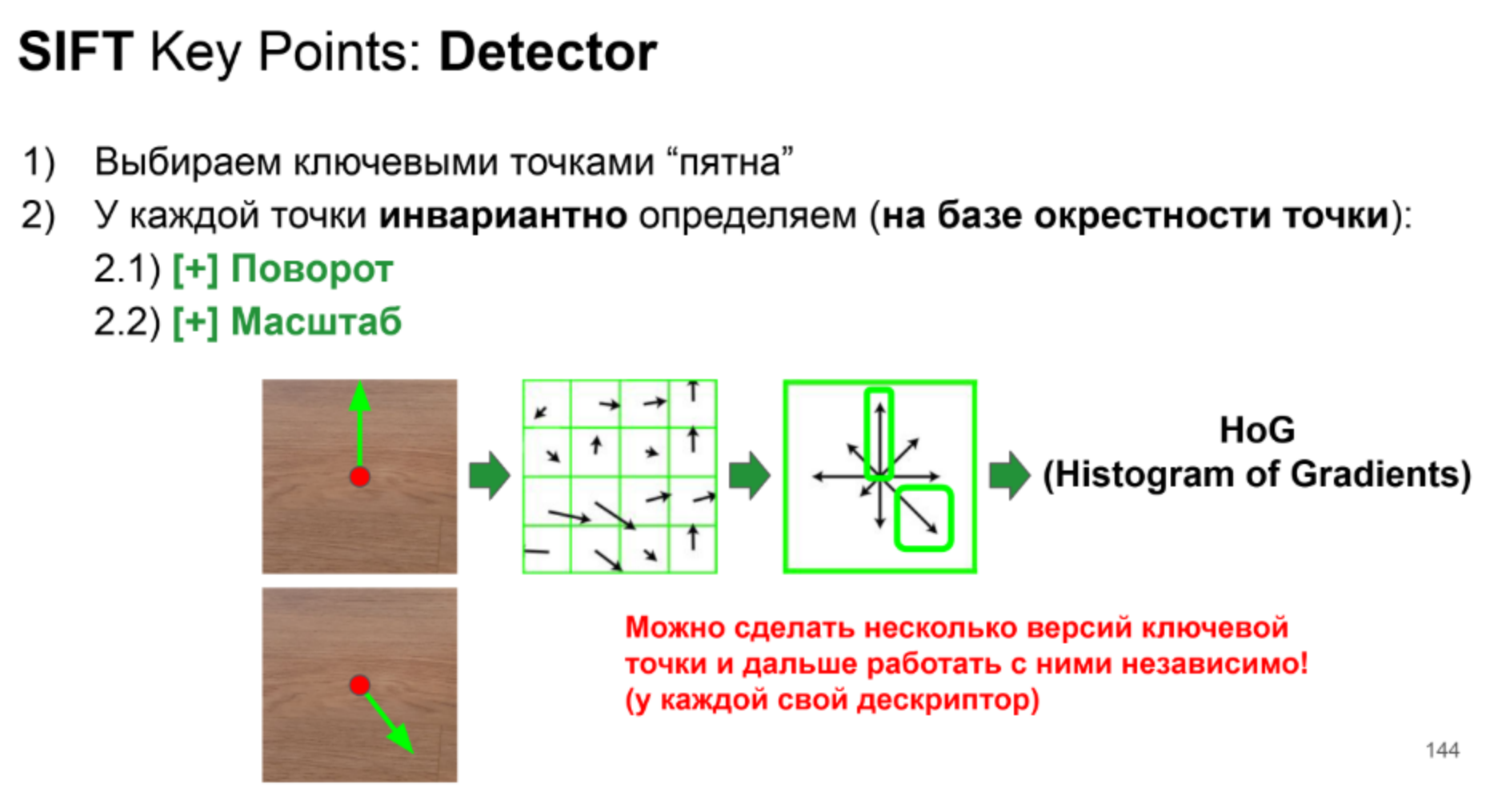
высоких частот не было ни сверху ни снизу
низкие частоты были и там и там - сократились
остались только **средние** частоты

убраны высокие частоты и средние частоты

Разложим картинку на частоты


ale
Gaussian
Difference of Gaussian (DOG)
Scale
сравниваем "на наличие яблока"
max в окрестности (3×3) (тут есть яблоко)
сравниваем на наличие яблока
если max среди 3×3×3 => особая точка

если это локальный максимум - здесь есть пятно, т.е. **ключевая точка**



SIFT Key Points: **Detector**

1) Выбираем ключевыми точками "пятна"
2) У каждой точки **инвариантно** определяем (**на базе окрестности точки**):
   2.1) **[+] Поворот**
   2.2) **[+] Масштаб**

HoG
(Histogram of Gradients)

Можно сделать несколько версий ключевой точки и дальше работать с ними независимо!
(у каждой свой дескриптор)

3) Для каждой точки строим дескриптор из HoG (гистограммы градиентов) по квадратной окрестности точки (повернутой и отмасштабированной)